Native T-SQL Language Service

Analysis, Metadata, Features, and SQL Scripting

Design, implementation, and validation review

For vscode-mssql PR #22860

Central question. How can vscode-mssql provide deterministic, low-latency T-SQL completion, diagnostics, hover, signature help, definition, folding, document symbols, and script generation over one immutable metadata generation—while remaining useful with partial or offline metadata, avoiding network I/O on an interactive request, and staying invisible until a later private-preview consumer explicitly composes it?

Assessment at the reviewed head. PR #22860 establishes a coherent source-only language and scripting library. Its contracts are VS Code-neutral; requests pin one immutable metadata view; completion and hover can request background hydration without awaiting it; diagnostics distinguish syntax truth from metadata-dependent claims; and routing contains native failures behind maturity gates, timeout, fallback, and circuit breaking. The branch adds no commands, settings, provider registration, controller, activation path, or extension bundle input. The focused 675-test corpus passes, full local build/lint and broad hosted suites pass, and all six platform launch checks are green at report freeze. Two small automated-review findings remain open and are documented rather than silently discounted.

Prepared for:	Karl Burtram
Primary change:	`dev/karlb/query_05_language_service` at `eafbbdc046`
Extraction base:	`95cf790e5`; 59 files, +25,750 / -0
Live PR base observed:	`53b707bbe`; mergeable, human review required
Metadata prerequisite:	merged PR #22836 / shared metadata service in `main`
Report source parent:	`bba79742e` in `kburtram/notes main`
Report snapshot:	31 August 2026, America/Los_Angeles

This is an as-built report. Historical refactor documents explain intent, but the pinned implementation, tests, PR state, and package artifacts control when the documents and code differ.

Contents

How to read this report

The report keeps implementation, intended invariants, executable evidence, and future integration separate:

Implemented
Present in the pinned PR head and traced to source.

Target invariant
A behavior or safety contract the library is intended to preserve.

Validated Exercised by a named test, build lane, package check, or observed repository state.

Open finding
A live review finding or evidence gap that remains at report freeze.

Future contract
A consumer, bridge, activation path, or maturity step not shipped by this PR.

Key takeaway

PR #22860 is not a replacement language service that users can turn on. It is the independently reviewable library beneath such a replacement: public data contracts, a tolerant analysis pipeline, native features, a pinned metadata adapter, strict scripting, routing/scheduling policies, observability, and tests. A later PR must supply a gated consumer and VS Code adaptation before any product behavior changes.

Evidence boundary

The strongest evidence is deterministic unit behavior and source-boundary inspection. Local packaging proves the tree packages; GitHub's package comparison more directly proves that the new source contributes zero size to the shipped VSIX at this head. Neither proves live-catalog semantic equivalence across the full T-SQL grammar. That is a later preview-graduation gate.

1 Executive summary

1.1 What the PR establishes

The change adds 37 production files under `src/sqlLanguage`, six under `src/sqlScripting`, fifteen focused unit files, and one observability classification edit. The architecture has four load-bearing properties:

1. **Pure request contracts and analysis.** Positions are zero-based UTF-16; payloads are JSON-safe; the core has no VS Code document or provider types.
2. **One truth generation per request.** Language work pins one immutable metadata view and reads readiness, environment, names, columns, parameters, keys, and definitions through that generation-stable seam.
3. **Honest partial behavior.** Absence and unavailability are different. The engine suppresses metadata-dependent claims, returns incomplete completion where appropriate, and preserves lexical/structural diagnostics when catalog validation is unavailable.

4. **Source-only private-preview boundary.** There is no package contribution, command, setting, activation event, controller, provider registration, Query Studio bridge, or AI consumer. Existing users cannot reach the code from this PR.

1.2 What the PR does not establish

It does not claim full T-SQL parsing, ScriptDOM equivalence, semantic-token or document-highlight support, live runtime integration, a production latency service-level objective, or a user-visible migration from the current SQL Tools Service language providers. The router includes a lazy bridge seam, but the bridge itself and the host composition that chooses it are deferred.

Private-preview meaning in this PR

No dedicated setting is added because there is no activated feature to gate. That is safer than a dead setting that suggests usable behavior. The first integration PR must require the umbrella experimental/private-preview flag *and* a dedicated feature-area flag, default both off, hide all contributed UI, and register runtime consumers only when the effective gate is true at activation.

2 Placement in the refactor train

PR #22860 is extraction PR05. Perftest, core observability, SQL Data Plane/STS2 foundation, and the metadata substrate are already in `main`; the Debug Console remains an independent UX track. PR05 builds directly from `main`, not from Debug Console PR03, and consumes only the accepted metadata APIs.

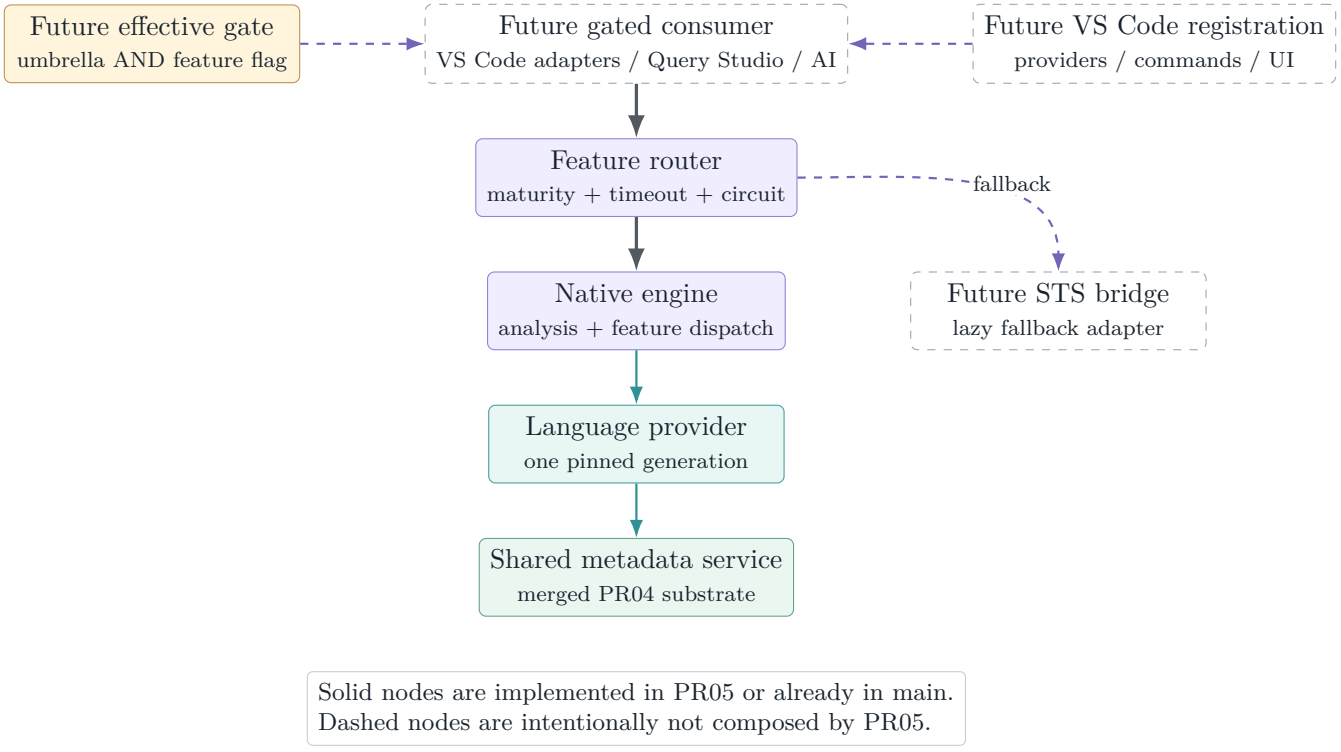

Figure 1: PR05 placement. The language library and metadata adapter exist, but every path from a user-visible consumer is still a future, gated composition step.

This placement is why the GitHub package comparison reports exactly the same MSSQL VSIX size for target and PR branches: 85,347 KiB in both. The extension bundler has no `sqlLanguage` or `sqlScripting` input at this head.

3 Public contracts and dependency boundaries

3.1 Consumer-neutral API

`sqlLanguage/api.ts` defines request and response data for completion, hover, signature help, diagnostics, definition, folding, document symbols, highlights, and semantic tokens. Every request carries text, version, URI, and position/range data without importing VS Code. This boundary lets a later host translate editor objects once, then test the engine with ordinary values.

The same file defines feature maturity and a capability table. The shipped native capability table marks completion, hover, signature, diagnostics, definition, folding, and document symbols as preview; highlights and semantic tokens remain off. Having types for a feature is therefore not the same as advertising an implementation.

Table 1: Native feature surface at the pinned head.

Feature	Capability	As-built behavior
Completion	Preview	Context/expectation-gated candidates, deterministic rank, partial-readiness honesty, lazy hydration kick.
Hover	Preview	Local, overlay, catalog, system, routine, built-in, and description facts when their readiness tier permits.
Signature help	Preview	Built-ins, catalog functions/procedures, nested calls, and `EXEC` argument tracking.
Diagnostics	Preview	T1 lexical/structural errors plus freshness-gated T2 208/207/209 binder warnings.
Definition	Preview	In-document local targets or virtual scripted catalog definitions with anchors and fidelity.
Folding	Preview	Batch, statement, comment, and region ranges.
Document symbols	Preview	Batch/statement structure and simple create/alter object declarations.
Highlights	Off	Interface reserved; native engine returns no result.
Semantic tokens	Off	Interface reserved; native engine returns no result.

3.2 Purity and import direction

Core analysis, feature computation, static data, provider interfaces, and scripting emitters are library code. Concrete host concerns live under `host`: metadata freshness, diagnostic journaling, timeout/yield behavior, and routing. The one sanctioned bridge from language metadata to the product catalog model is `provider/catalogProvider.ts`.

Design invariant

Interactive feature code consumes a synchronous `IPinnedMetadataView`; it does not hold a mutable store, lease, network client, VS Code document, or STS service. Definition text is the one deliberate async lookup on the pinned view, and strict scripting asks the host to establish freshness before it creates the engine.

4 Document analysis pipeline

4.1 From text to reusable structure

The native engine analyzes one immutable text snapshot, then reuses structural work across feature calls when document version and text length match. The stages are deliberately tolerant:

1. **Text snapshot** maps offsets to zero-based UTF-16 line/character positions and back, including ordinary LF/CRLF and mixed input.
2. **Lexer** recognizes T-SQL words, identifiers, punctuation, comments, strings, variables, SQLCMD-sensitive regions, and `GO` / `GO n` line semantics.
3. **Segmenter** partitions batches and statements while respecting module bodies and rule-sensitive constructs.
4. **Sketch parser** extracts the structure features need: query scopes, sources, CTEs, select items, declarations, `EXEC`, DML targets, and table declarations. It remains total on incomplete editor text.
5. **Overlay** models script-local state such as temporary tables, declared table variables, scripted tables, and batch-scoped variables.
6. **Binder and cursor expectation** combine local structure, the effective database, and one pinned metadata view for a particular feature request.

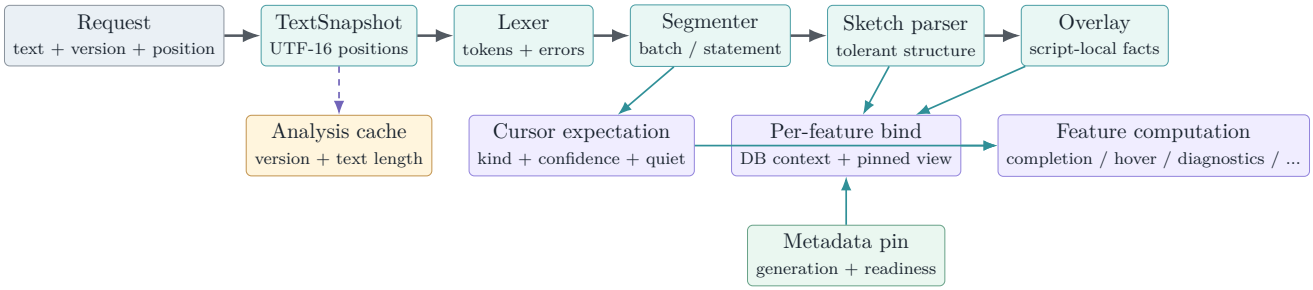

Figure 2: As-built analysis pipeline. Syntax structure is cached per document analysis; metadata is pinned and binding is performed for each feature request so one response cannot mix generations.

4.2 Why a sketch instead of a full AST

Editor input is commonly unfinished. A full grammar can make a single missing token collapse useful context; an unbounded hand-written parser can accidentally become a second compiler. The sketch deliberately extracts only what current features consume and records uncertainty. The cursor-expectation layer improves completion category selection and suppresses suggestions in declarations, data-type positions, comments, strings, and other quiet contexts, but it is not the richer parser-v2/AST envisioned by later design notes.

Semantic ceiling

The tolerant pipeline is broad, not complete. Dynamic SQL, unusual procedural forms, deeply dialect-specific syntax, and ambiguous recovery can exceed the sketch. The correct failure mode is suppression, incomplete results, or fallback—never a confident catalog error manufactured from an untrusted parse.

5 Metadata contract and honest degradation

5.1 Generation-stable provider seam

`provider/types.ts` separates a mutable provider from the immutable view used by a request. The provider exposes current generation, environment, readiness, a pin operation, databases, an explicit hydration request, and a change event. The pin exposes synchronous resolution and projection over an opaque `LangObjectRef`; consumers cannot retain mutable catalog objects or invent cross-generation identity.

Readiness is sectioned across objects, columns, parameters, foreign keys, and definitions. The global mode—full, lite, partial, or offline—does not erase section truth. Resolution is a tagged result: resolved, default-schema choice, ambiguous, not found, or unavailable. This prevents the dangerous equation "not returned = does not exist."

5.2 Adapter over the shared metadata service

`catalogProvider.ts` is the only production adapter from the merged metadata substrate to language types. It maps a compact immutable `CatalogSnapshot` into the pinned language surface, carries database/case/capability context, and returns honest unknown states when no snapshot exists.

When a feature discovers that object names are ready but columns are not, the adapter may request hydration. The current implementation coalesces to one in-flight refresh, avoids starting while already loading/stale, makes at most one kick per generation, and enforces a 30-second floor. The present metadata service refreshes the full ladder; the language seam is already shaped so later section-level hydration does not change feature contracts.

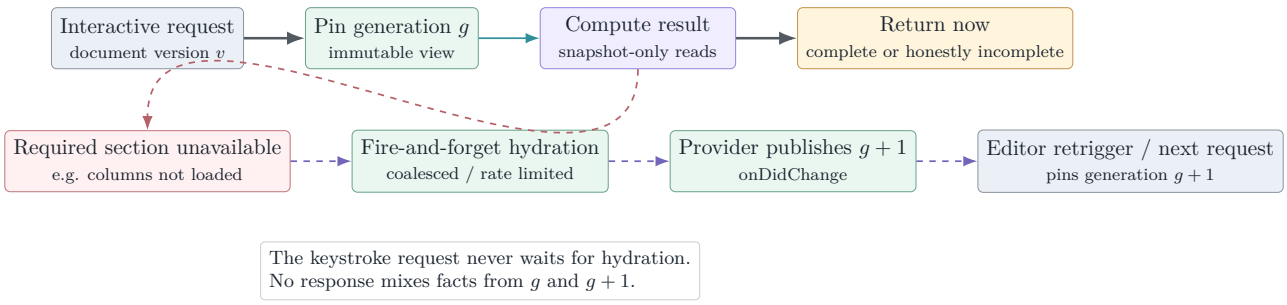

Figure 3: Lazy hydration without hot-path I/O. The current request finishes from generation g; a later request observes $g + 1$.

5.3 System and offline views

The null provider supplies explicit offline readiness and no false catalog facts. Fixture providers simulate full, partial, and lazy-hydration states for deterministic tests. The system-catalog decorator supplies a curated names-only fallback for `sys` and `INFORMATION_SCHEMA`; live catalog facts win, identifiers are synthetic and generation-local, and the fallback does not claim column types, nullability, keys, definitions, or other facts it does not carry.

No network on the feature path

Completion, hover, signature help, and ordinary diagnostics read the pin synchronously. A hydration request is advisory and asynchronous. Strict scripting is different by design: its host establishes an explicit freshness verdict before constructing the pinned engine. The separation makes latency and truth policy visible rather than allowing an accidental catalog call inside fuzzy matching or binding.

6 Language feature behavior

6.1 Completion

Completion first classifies cursor expectation, then uses the legacy context classifier and bound statement facts to choose producers. Producers cover statement keywords/defaults/snippets, member access, table sources, joins and foreign-key predicates, expressions and columns, variables, built-ins, star expansion, `INSERT`, `UPDATE`, `EXEC`, `DECLARE`, `USE`, data types, and databases.

Candidate selection is deterministic: category gates run before fuzzy matching; scores are stable; ordinal comparison breaks ties; output is capped at 600. Partial metadata does not produce an authoritative empty list. Member-access on an object with unloaded columns returns an incomplete reason and can kick background hydration.

6.2 Diagnostics

Diagnostics intentionally separates two truth tiers:

- **T1 lexical/structural diagnostics** come from the text, tokens, segments, and tolerant statement structure. They include lexical faults, malformed `GO`, bracket/parenthesis/statement structure, unrecognized statements, and recoverable `SELECT` syntax.
- **T2 binder diagnostics** depend on catalog trust: invalid object 208, invalid column 207, and ambiguous column 209. A host freshness verdict of `notValidated`, partial readiness, dynamic/unsupported syntax, uncertain database context, or other recorded suppression reason prevents these warnings.

The diagnostic result records suppression counts/reasons so "quiet because valid" can be distinguished from "quiet because the engine lacked authority." Whole-document work is resumable by statement/work unit for scheduler time slicing.

6.3 Hover and signature help

Hover resolves schema/database/name-chain parts, aliases, projected aliases, CTEs, derived tables, temporary/script tables, table variables, variables and parameters, catalog routines, and built-ins. H7 descriptions appear only when description readiness is true. Cross-database and `USE` handling share the database-context map used by binding.

Signature help scans the enclosing call without requiring a complete parse. It understands nested and grouped built-in calls, catalog functions/procedures, and `EXEC` argument positions. Both features stay quiet in comments, strings, SQLCMD regions, and positions where the scanner cannot make an honest call.

6.4 Definition, folding, and symbols

Definition has two destinations. Local aliases, variables, CTEs, temporary objects, table variables, and derived columns return in-document ranges. Catalog objects or columns route through the scripting service and return virtual content plus semantic anchors. Folding and document symbols use only the analysis snapshot and do not require catalog I/O.

Table 2: Feature truth dependencies and degradation.

Feature	Primary truth input	When truth is incomplete
Completion	Cursor expectation, sketch, overlay, objects/columns/keys	Limit categories, return incomplete, request hydration; retain static/built-in/local candidates.
Diagnostics	Tokens/structure; then bound objects/columns	Preserve T1; suppress affected T2 and account for the reason.
Hover	Local structure or specific catalog sections	Return only facts whose section is ready; no invented type/detail.
Signature	Call scanner, built-ins, routine parameters	Built-ins/local syntax remain; catalog signatures require parameter readiness.
Definition	Local declarations or scriptable catalog target	Local range when known; virtual script with provenance; unavailable/refusal when definition truth is insufficient.
Folding/symbols	Snapshot, tokens, segments, sketch	Continue structurally; no metadata dependency.

7 Scheduling, routing, and failure containment

7.1 Sliced diagnostics scheduler

`host/scheduler.ts` is VS Code-free. A document or metadata change restarts a default 300 ms debounce. The pass then performs work in default 8 ms slices and yields by macrotask between slices. Its stamp normally combines document version and metadata generation; the scheduler rechecks the stamp before starting, after an asynchronous pass factory returns, between slices, and before publishing.

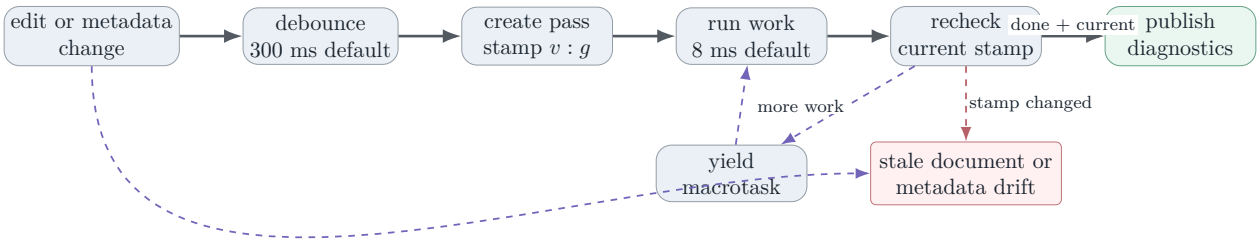

Figure 4: Diagnostics scheduling. Old results are abandoned instead of being published after either document edits or metadata-generation drift.

7.2 Feature router

`host/router.ts` centralizes rollout and fallback. A consumer supplies an engine preference, capability table, rollout threshold, lazy bridge factory, and optional diagnostics observer. Native work is eligible only when preference and maturity allow it. A default 5-second timeout contains a hung native call; after two consecutive failures for a feature, its circuit opens and later calls go directly to fallback until reset. Failures are isolated per feature, so diagnostics cannot disable completion.

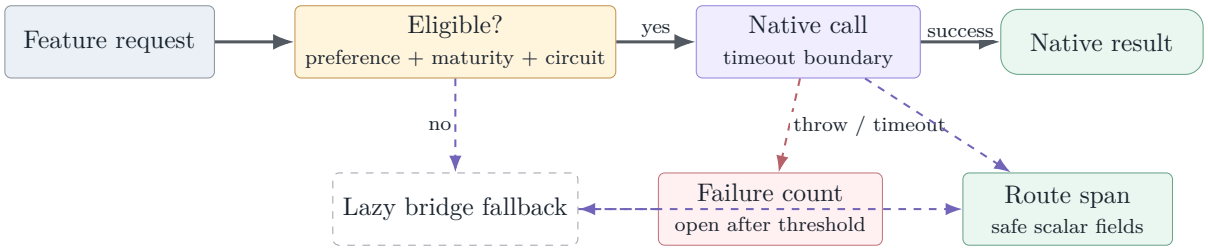

Figure 5: Per-feature routing. The native library can be introduced incrementally while retaining a lazy fallback and containing repeated failures. The bridge and composition are not part of PR05.

8 Definition and SQL scripting

8.1 Scripting contract

`sqlScripting/api.ts` defines create, alter, create-or-alter, drop, drop-and-create, select-top, insert, update, delete, and execute operations. Results carry script text, anchors, fidelity, notes, unavailability reason, and metadata provenance.

Fidelity is explicit:

- **F2** is definition-level output: a stored module definition or a table statement with the required detailed metadata.
- **F1** is structurally useful but incomplete output, such as a table definition degraded by missing optional details.
- **F0** is a template, notably DML scaffolding, and is never presented as a faithful reconstruction.

Module scripting uses token-level head rewriting so create/alter/create-or-alter changes do not replace keywords inside comments, strings, or bodies. Table scripting synthesizes columns, types, defaults/computed expressions, identity, and constraints only when corresponding metadata is ready. DML emitters produce honest templates. The writer records semantic anchors so definition navigation can land on the object, parameter, column, or statement rather than merely opening line zero.

8.2 Strict freshness flow

The concrete scripting host calls the metadata lease's strict policy before it pins. A host-side timeout is a backstop around the metadata policy timeout. Rejection, timeout, or an unavailable strict verdict refuses the operation; explicit offline-snapshot policy can instead allow cached output with provenance and an honest banner. This makes scripting stricter than interactive completion because generated DDL is more likely to be copied and executed.

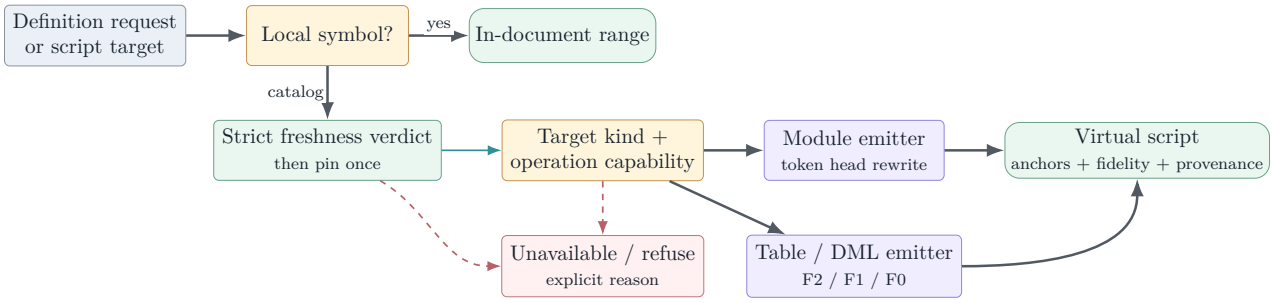

Figure 6: Definition and scripting truth flow. Local navigation stays in the document; catalog navigation produces a provenance-bearing virtual script or an explicit refusal.

9 Observability and privacy

The engine emits bounded spans for lexing, segmentation, parsing/sketching, completion, hover, signature help, diagnostics, definition, routing, and scripting. `perfTelemetry.ts` classifies `sqlLanguage.*` and `sqlScripting.*` under the existing `sqlLanguage` feature family; the accepted observability registry already covers those prefixes.

Normal diagnostics use safe scalar measures such as duration, counts, route, readiness, cache/hydration state, suppression reasons, and result shape. Rich completion diagnostics can classify prefix/name-part/top-label values as `user.text` or `object.name`; the shared diagnostics layer digests these by default and exposes them only under the explicitly elevated, time-bounded capture policy. The privacy claim is therefore not "identifiers never enter an event object." It is that sensitive fields are classified at the call site and governed by the accepted capture/redaction boundary.

Observability contract

Do not add raw document text, SQL batches, connection strings, result rows, or unclassified object names to a language span. New attributes must be registered and typed before emission. Performance evidence should use counts, durations, readiness and suppression state; diagnosis that truly needs content must remain an explicit elevated-capture operation.

10 Test design and validation

10.1 Focused semantic corpus

The 15 focused unit files execute as 77 suites and 675 tests at the pinned head: 675 passed, zero failed, zero skipped, with 1.325 seconds of test-body time recorded in the hosted JUnit XML. The value is not just volume; the suites target incomplete syntax, partial truth, deterministic ordering, stale publication, and refusal behavior.

Table 3: Focused language and scripting inventory.

Area	Tests	Principal invariants
Completion	78	Member access, table sources, joins/FKs, expressions, CTE/derived/temp objects, DML/EXEC/DECLARE/USE, DDL declarations, star expansion, partial-readiness honesty, latency budget, statement starts, static system catalog.
Cursor expectation	17	Category/confidence selection and quiet positions before completion producers run.
Core / router	20	Lexer, segmenter, feature routing, maturity, fallback/circuit behavior, and LS-0 surface.
Definition	56	Aliases, variables, CTE/temp/script-local objects, catalog table/module scripts, honesty ladder, and capability routing.
Diagnostics	168	Lexical, `GO`, structural and recovery T1; 208/207/209 T2; 62-case zero-unexpected-diagnostic honesty corpus; suppression accounting; lazy hydration; engine pass.
Diagnostics freshness	6	Freshness gate and scheduler drift classification.
Hover	124	Catalog and local symbol families, H7 descriptions, partial truth, `USE`/cross-database/MERGE, DML positions, never-hover contexts, and system catalog.
Journal	3	Normal versus elevated-capture diagnostic fields.
Providers	13	Fourslash fixture parsing, fixture catalog behavior, and offline catalog-adapter honesty.
Scheduler	9	Debounce, sliced execution, stale document cancellation, metadata drift, and asynchronous factory recheck.
Signature	60	Built-ins, nesting/grouping, catalog routines, `EXEC`, and honesty positions.
Sketch	20	Total structural modeling across supported statement families and incomplete text.
System catalog	15	Static data helpers, live-over-fallback precedence, and diagnostic interaction.
SQL scripting	75	Module rewrites/anchors/honesty; table F2/F1/anchors; DML; capabilities/routing; provenance/offline banners.
Strict scripting host	11	Freshness policy, timeout/refusal, pin ordering, provenance, and explicit offline behavior.
Total	**675**	**Zero failures and zero skips in the focused selection.**

10.2 Broader local validation

Validation was run after merging extraction-base `95cf790e5` into the topic branch:

Table 4: Independent local validation at `eafbbdc046`.

Lane	Result	Interpretation / exclusion
Root build	Pass	Full monorepo build: toolkit, MSSQL extension/typecheck/bundles/webviews, SQL Projects, and Data Workspace.
Root lint	Pass	Repository lint completed without errors.
Focused language/scripting	675 pass; 0 fail	All 77 suites in table 3; no skipped cases.
Observability contracts	7 pass; 0 fail	Prefix classification and accepted diagnostics registry behavior.
Broad MSSQL hosted tests	4,710 pass; 0 fail; 10 skip	Inverse selection excludes only two independently documented current-main Windows baseline suites: `Utility Tests - Path handling` and `QueryCompletionSoundService`.
SQL Projects hosted tests	205 pass; 0 fail; 6 skip	Executed in its separate VS Code host.
Localization / pre-commit	Pass	Extraction, formatting hooks, and diff checks clean; this PR adds no product strings.
Production online package	Pass	Released STS `6.0.20260827.1`; local artifact 1,542 files / 99.45 MiB. Metafile contains no new language/scripting bundle input.

10.3 GitHub validation snapshot

At report freeze, GitHub reports the PR open, mergeable, non-draft, labeled `Automated Review`, and still requiring human review. Build-and-test, actions and JavaScript/TypeScript analysis, CodeQL, normalization, CLA, package creation, and all six Linux/macOS/Windows x64/arm64 VSIX-launch jobs are green. The separate Azure `VSCode MSSQL - Pull Request` context remains pending.

The GitHub package comparison is stronger than the absolute local size for this scope question: target and head are both 85,347 KiB for MSSQL, with zero delta for every packaged extension. Codecov reports 94.34331% patch coverage with 1,041 changed lines uncovered and project coverage rising from 78.22% to 79.86%. The largest uncovered concentration is the production metadata adapter; hover, diagnostics, definition, completion, sketch/context, cursor expectation, engine, and text snapshot also retain uncovered branches.

Evidence boundary

High patch coverage and 675 focused passes support the implementation, but the important properties are semantic: no mixed-generation answer, no false “not found” from unavailable metadata, no stale diagnostic publication, deterministic ranking, and strict scripting refusal. Coverage percentage alone cannot prove these. Conversely, zero bundle-size delta proves inert packaging, not live feature quality.

11 Known limitations and live review state

11.1 Automated-review findings still open

Copilot reviewed 46 of 59 files and left two unresolved threads at `eafbbdc046`:

1. `sqlScripting/scriptWriter.ts` counts `\n` while tracking anchors but does not treat a standalone carriage return as a newline. LF and CRLF output are handled, but CR-only or mixed CR-only

content could produce incorrect anchor line/character values. This is a narrow compatibility edge, not evidence that ordinary generated CRLF scripts are mis-anchored.

2. `sqlScripting/scriptingService.ts` says synonym targets are "not hydrated" when the real limitation is that this engine has no scriptable synonym definition. The behavior is an honest refusal; the explanatory note is imprecise.

These findings should be dispositioned in the PR before merge. They do not change the architecture described here, but this report does not mark the review closed while the threads remain open.

11.2 Deliberate implementation limits

- The sketch is not a full T-SQL AST and the feature set is not a ScriptDOM equivalence claim.
- Highlights and semantic tokens are off.
- The bridge/fallback adapter is a router seam, not an implementation in this PR.
- The metadata adapter's lazy kick currently refreshes the full metadata ladder; section-specific hydration remains a future optimization.
- The system fallback is names-only and deliberately cannot support type/detail/key/definition claims.
- No live editor consumer, settings hierarchy, provider registration, Query Studio integration, AI completion, SQL document service, sqlcmd preprocessing, execution batch splitter, or query execution helper ships here.
- Unit fixtures cover a wide semantic corpus, but real-server collation, permissions, catalog scale, dynamic SQL, uncommon grammar, Azure/serverless behavior, and remote extension-host performance remain preview validation work.

11.3 Branch-drift boundary

The topic includes `main` through extraction base `95cf790e5`. At report freeze the live PR API identifies a later base `53b707bbe`, and the locally fetched `origin/main` has advanced again to `e0f18fd8d`. GitHub currently calculates the PR as mergeable, but a later main refresh should repeat the full build/lint/focused/broad/package gates before the final merge decision.

12 Integration and graduation plan

12.1 Next implementation step

The next PR should be a gated consumer, not another implicit activation edit scattered across the extension. It should:

1. add one composition root that adapts VS Code documents and result types to the consumer-neutral contracts;
2. require `mssql.enableExperimentalFeatures` and a dedicated default-off native-language/AI feature flag before registration;
3. hide contributed commands and UI with complete manifest `when` clauses, not only disabled command enablement;
4. choose native versus existing behavior through the router, provide the real lazy bridge, and preserve per-feature fallback;

5. wire the metadata provider from the accepted shared catalog service and dispose leases/change subscriptions correctly;
6. carry perftest and scenario tests for activation-off invisibility, gated activation, latency, fallback, and stale-result suppression; and
7. avoid coupling AI completion to a global language-provider cutover—AI can be an early gated consumer of the same analysis/provider seam.

12.2 Graduation gates

Before moving from private preview toward default-on, validation should add:

1. a live-SQL truth corpus across supported versions, Azure variants, collations, permissions, synonyms, temporary/script objects, and cross-database contexts;
2. side-by-side native/STS result comparison with categorized differences rather than raw count parity;
3. interactive p50/p95/p99 budgets for completion/hover/signature and sliced whole-document diagnostics across document/catalog size tiers;
4. remote and web/virtualized extension-host CPU/memory measurements plus metadata hydration/retrigger behavior;
5. UI-level tests for flag hierarchy, command/menu invisibility, reload semantics, circuit-breaker fallback, and disabling the preview; and
6. an explicit privacy audit of elevated rich fields, journal/export behavior, and any AI consumer payload boundary.

Table 5: Mechanism-to-evidence traceability.

Mechanism	Current evidence	Next proof
Pure analysis contracts	Import shape, fixture engines, hosted unit tests	Consumer adapter tests in the first activation PR.
Pinned-generation metadata	Provider types, catalog/fixture/null/system tests	Forced generation change during live feature requests.
No hot-path network wait	Synchronous pin contract and lazy kick tests	Instrumented live provider/perftest assertion.
Completion determinism	Category/rank/ordinal and fixture tests	Cross-platform replay corpus and large-catalog latency.
Diagnostic honesty	T1/T2 split, 62-case honesty suite, freshness/drift tests	Real-server truth corpus and native-vs-STS differential run.
Stale-result suppression	Deterministic scheduler stamp tests	VS Code integration under edit/hydration storms.
Strict scripting provenance	Freshness/refusal/offline and emitter tests	Live permissions/encryption/definition corpus.
Failure containment	Router maturity/timeout/circuit unit behavior	Actual bridge and consumer-level fallback telemetry.
Private-preview invisibility	No manifest/controller/bundle inputs; zero package delta	Manifest/runtime gate tests when activation is introduced.

13 Final assessment

Key takeaway

PR #22860 is a strong independent extraction boundary. It turns the earlier language-service design into a concrete, testable library without prematurely changing the product. The most important choices are sound: tolerant analysis over incomplete editor text; generation-pinned metadata; explicit readiness and suppression; deterministic completion; sliced, stale-safe diagnostics; provenance-bearing scripting; and per-feature fallback containment.

The branch is ready for focused human design review and final automated-thread disposition, not for default-on product exposure. Its remaining uncertainty is mostly the expected kind for a source-only foundation: live integration, large and unusual real catalogs, full grammar breadth, performance at product scale, and UX-gated activation. Those should be measured by the owning consumer PRs rather than blurred into this extraction.

Decision statement

Recommended disposition at this snapshot: preserve the architectural boundary; resolve or explicitly decline the two small open review threads; refresh from current `main` if required; rerun the stated gates; then proceed to human merge review. Do not expose a command, provider, setting-driven cutover, Query Studio surface, or AI completion path until a later PR supplies the full private-preview gate and integration tests.

Evidence and source map

Reproducibility note

This report is pinned to an active PR. Review threads, Actions conclusions, main-branch refs, coverage, and mergeability are observations at the stated snapshot. Re-fetch the head and live PR state after any code or base-branch change before reusing its final disposition.

Evidence class	Primary sources used
PR state	PR #22860, head `eafbbdc046`, extraction base `95cf790e5`, live API base `53b707bbe`, review threads, Actions check rollup, package-size bot, and Codecov report.
Public API / host	`extensions/mssql/src/sqlLanguage/api.ts`; `host/nativeEngine.ts`; `host/router.ts`; `host/scheduler.ts`; `host/scriptingHost.ts`.
Analysis core	`core/text/textSnapshot.ts`; `core/lexer.ts`; `core/segmenter.ts`; `core/sketch/*`; `core/parser/cursorExpectation.ts`; `core/binder.ts`; `core/overlay.ts`; database/name/fuzzy/quote helpers.
Language features	`features/completion.ts`; `diagnostics.ts`; `hover.ts`; `signatureHelp.ts`; `definition.ts`; `folding.ts`; `documentSymbols.ts`; generated keyword/built-in/default/system data.
Metadata seam	`provider/types.ts`; `catalogProvider.ts`; `systemCatalogView.ts`; `nullProvider.ts`; `fixtureProvider.ts`; merged metadata-service implementation from PR #22836.
SQL scripting	`extensions/mssql/src/sqlScripting/api.ts`; `scriptingService.ts`; `scriptWriter.ts`; module/table/DML emitters; strict host under `sqlLanguage/host`.
Observability	`extensions/mssql/src/perf/perfTelemetry.ts`; accepted diagnostics registry and privacy contracts already in `main`; journal tests.
Tests	Fifteen `extensions/mssql/test/unit/sqlLanguage*.test.ts` and `sqlScripting*.test.ts` files; hosted JUnit XML; independent build/lint/broad-suite/package record summarized in table 4.
Design intent	`vscode-ext-refactor/language-service-docs/05-tsql-language-service-design.md`; `current_completions_parser_design.md`; `PROGRESS.md`; proposed diagnostics/completion designs. Code controls where historical/future design differs from the extraction.
Private-preview policy	`C:/repos/work2/resume.md` and the accepted PR04 preview-gating implementation; no PR05 manifest or activation additions.